EL KATTOUFI Youssef

Année 2026 – 2027

TP 1 MCU



Etablissement scolaire : Polytech Sorbonne

Formation : Electronique et Informatique

Matières : Microcontrôleur 2

Enseignant :Francis BRAS

Table des matières

Remerciements 5

I. Introduction et objectif du TP 6

II. Présentage du montage 6

III. Bilan des entrées / sorties 7

A. Bilan E/S - Carte Nucleo STM32 F103RB 7

B. PCF8574 et des LED 7

IV. Explication du programme 9

A. Configuration des GPIO — GPIO_InitPin() 9

B. Temporisation — delay_us() et delay_ms() 9

C. Gestion du ZIF16 — ZIF_Config() et ZIF_Lire() 9

D. I^2C en bit banging — I2C_Start(), I2C_Stop(), I2C_EcrireBit(), I2C_LireBit() 10

E. Dialogue avec les PCF8574 — PCF_Ecrire() et PCF_Lire() 10

F. Casiers — Casiers_Initialiser(), Casiers_LEDs() et Casiers_Lire() 10

G. USART2 et réception des trames — UART_Init() et USART2_IRQHandler() 10

H. Transmission — FIFO_AjouterReponse() et FIFO_Emettre() 10

I. Séquence de démarrage des LED — Allumer_Toutes_LEDs(1) 10

J. Test local des DIP-switchs — Maj_LEDs_Casiers() 11

K. Boucle principale — main() 11

L. Récapitulatif de toutes les fonctions de main.c 11

V. Configuration périphérique STM32 13

A. GPIO 13

B. Configuration USART2 14

VI. Configuration et vérification du fonctionnement du système 14

A. Configuration des périphériques 14

B. Tests et validation des interrupteurs 15

C. Vérification des niveaux logiques du ZIF16 16

VII. Le logiciel processing 16

VIII. Petite explication concernant l 18
A. DIP-switchs 18
B. Première lecture et vérification du ZIF16 19
IX. Commandes des LED et lectures des DIP-switchs 20
A. Commande envoyée depuis Processing 20
B. Commandes et lecture sur le bus I^2C 20
C. Correspondance avec les commandes du code 22
X. Configuration et lecture des 16 broches du ZIF16 22
XI. Difficultés rencontrées et assistance IA 24
XII. Conclusion 25
Tables des figures 27

Remerciements

Je tiens tout d'abord à remercier Monsieur Francis BRAS pour avoir conçu et mis en place un TP aussi stimulant, qui nous a amenés à réfléchir, à nous questionner et à surmonter de nombreuses difficultés techniques.

Je souhaite également le remercier pour sa disponibilité et son accompagnement tout au long du projet, notamment à travers ses échanges sur Discord, où il a toujours su nous apporter son aide et ses conseils lors des phases de programmation.

Enfin, je le remercie pour la mise à disposition du matériel nécessaire à la réalisation de ce projet, ainsi que pour la transmission des connaissances en Microcontrôleur 2.

Son investissement et son suivi nous ont permis de mettre en pratique nos connaissances tout en développant notre autonomie et notre capacité à résoudre des problèmes techniques.

I. Introduction et objectif du TP

Dans ce TP, le but était de réaliser une partie d'un système de rangement pour les composants logiques de la série 4000. Chaque composant doit pouvoir être rangé dans le bon casier et testé sur un support ZIF16 pour vérifier son fonctionnement. Le STM32F103RB sert donc à faire le lien entre les composants, les voyants des casiers et l'ordinateur.

Pour la maquette, j'ai travaillé avec six LED bicolores rouge/vert réparties sur deux PCF8574, ainsi qu'un troisième PCF8574 pour lire les interrupteurs qui simulent l'ouverture et la fermeture des casiers. L'énoncé prévoit 16 casiers au total, mais le montage réalisé est réduit au matériel disponible. Le principe reste le même.

J'ai réalisé les essais progressivement : configuration des GPIO, communication I^2C en logiciel, commande des LED, lecture des interrupteurs, puis échange de trames entre le STM32 et Processing sur le PC.



Figure 1 : Visualisation Keil µVision + Visual Studio Code

Remarque :

Comme à mon habitude le projet se trouve sur mon compte github personnel à l'adresse suivante : https://github.com/Darkzveller/TP1_MCU

II. Présentage du montage

La carte NUCLEO STM32-F103RB joue le rôle de contrôleur principal. Elle dialogue avec les PCF8574 sur un bus I2C émulé par logiciel et avec l'ordinateur grâce à l'USART2 relié au port USB ST-LINK. Le connecteur ZIF16 utilise directement seize broches du STM32.

Pour gagner du temps, j'ai aussi réalisé le câblage du TP2 sur la plaque Labdec pendant que j'avançais sur le TP1. Je faisais donc les deux TP en parallèle. C'est pour cela que certaines photos du montage pourront faire apparaître des fils ou des composants liés au TP2, qui ne font pas partie de la fonction casiers décrite ici.

III. Bilan des entrées / sorties

A. *Bilan E/S - Carte Nucleo STM32 F103RB*

Nom module	Nom signal	in/out/ ana/num	Pull up / Pull down	Périphérique	PX_n	Commentaires
Casier	CLK_PCF	OUT / Num	Pull up	GPIOB	PB6	Horloge I2C logiciel des PCF8574
Casier	SDA_PCF	IN/OUT / Num	None	GPIOB	PB7	Transmission et réception I2C
COM_ORDI	GPIOA, 2 (TX)	OUT / Num	None	USART2	PA2	Envoi des données vers Processing
COM_ORDI	GPIOA, 3 (RX)	IN / Num	None	USART2	PA3	Réception des commandes de Processing
TESTEUR	GPIOC, 0	IN/OUT / Num	Pull up / Pull down	GPIOC	PC0	Patte 1 du ZIF16
TESTEUR	GPIOC, 1	IN/OUT / Num	Pull up / Pull down	GPIOC	PC1	Patte 2 du ZIF16
TESTEUR	GPIOC, 2	IN/OUT / Num	Pull up / Pull down	GPIOC	PC2	Patte 3 du ZIF16
TESTEUR	GPIOC, 3	IN/OUT / Num	Pull up / Pull down	GPIOC	PC3	Patte 4 du ZIF16
TESTEUR	GPIOC, 4	IN/OUT / Num	Pull up / Pull down	GPIOC	PC4	Patte 5 du ZIF16
TESTEUR	GPIOC, 5	IN/OUT / Num	Pull up / Pull down	GPIOC	PC5	Patte 6 du ZIF16
TESTEUR	GPIOC, 6	IN/OUT / Num	Pull up / Pull down	GPIOC	PC6	Patte 7 du ZIF16
TESTEUR	GPIOC, 7	IN/OUT / Num	Pull up / Pull down	GPIOC	PC7	Patte 8 du ZIF16
TESTEUR	GPIOC, 8	IN/OUT / Num	Pull up / Pull down	GPIOC	PB8	Patte 9 du ZIF16
TESTEUR	GPIOC, 9	IN/OUT / Num	Pull up / Pull down	GPIOC	PB9	Patte 10 du ZIF16
TESTEUR	GPIOC, 10	IN/OUT / Num	Pull up / Pull down	GPIOC	PB10	Patte 11 du ZIF16
TESTEUR	GPIOC, 11	IN/OUT / Num	Pull up / Pull down	GPIOC	PB11	Patte 12 du ZIF16
TESTEUR	GPIOC, 12	IN/OUT / Num	Pull up / Pull down	GPIOC	PB12	Patte 13 du ZIF16
TESTEUR	GPIOC, 13	IN/OUT / Num	Pull up / Pull down	GPIOC	PB13	Patte 14 du ZIF16
TESTEUR	GPIOC, 14	IN/OUT / Num	Pull up / Pull down	GPIOC	PB14	Patte 15 du ZIF16
TESTEUR	GPIOC, 15	IN/OUT / Num	Pull up / Pull down	GPIOC	PB15	Patte 16 du ZIF16

Figure 2 : Bilan E/S - Carte Nucleo STM32 F103RB

Cela représente vingt broches du STM32 : deux pour l'I^2C logiciel, deux pour l'USART2 et seize pour le testeur ZIF16. Les LED et les contacts sont raccordés sur les PCF8574 et non directement sur des broches supplémentaires du STM32.

B. *PCF8574 et des LED*

Les broches A0, A1 et A2 servent à choisir l'adresse de chaque PCF8574. Il faut donc placer les cavaliers correctement pour éviter que deux composants aient la même adresse.

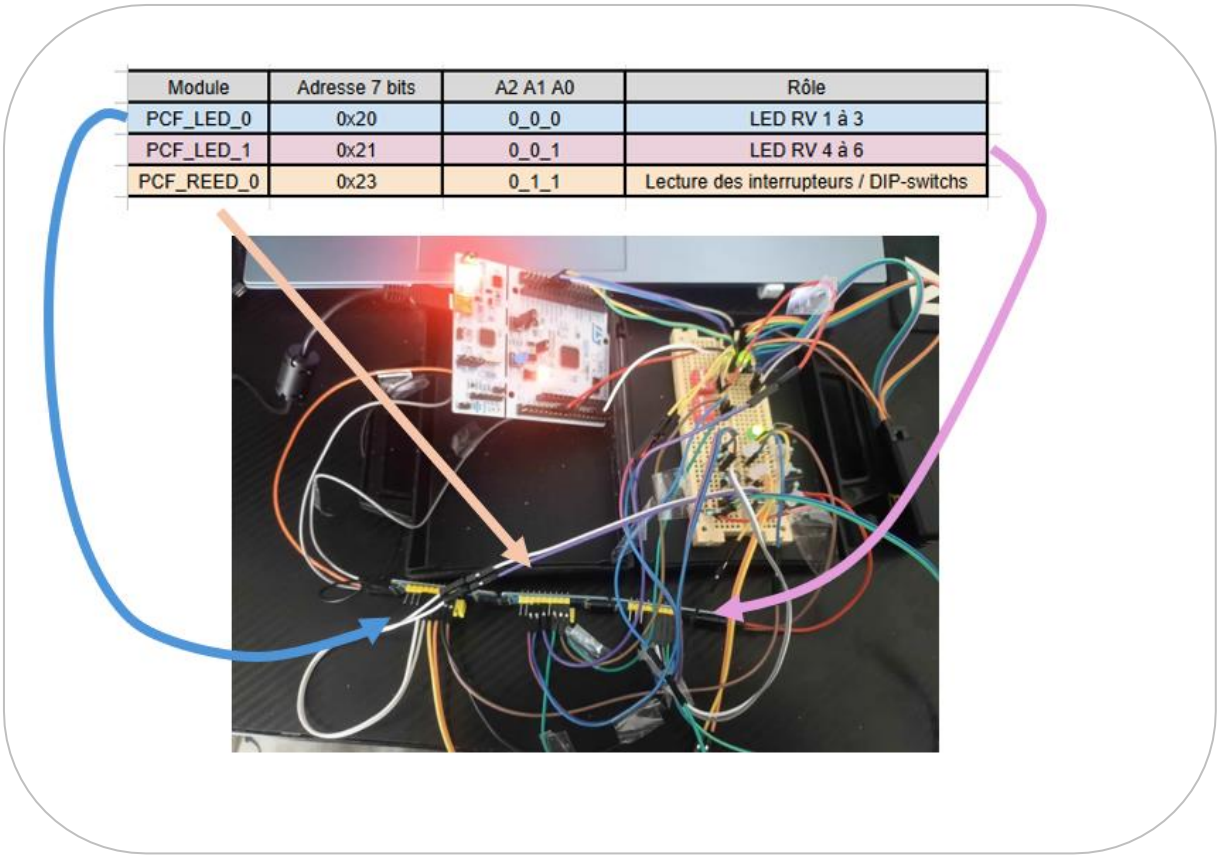


Figure 3 : Localisation des composants et configuration des modules

Attention, les commentaires écrits à côté des #define PCF_LED_0 et PCF_LED_1 dans main.c sont contradictoires avec les calculs. Les adresses à retenir sont bien les valeurs évaluées par le code : 0x20, 0x21 et 0x23.

Pour le PCF qui vient lire l'état d'ouverture du casier, j'ai effectué le câblage suivant :

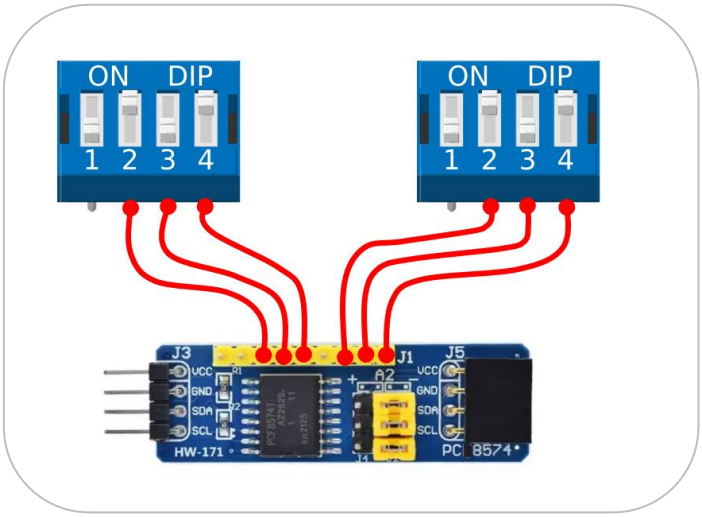


Figure 4 : Câblage spécifier du PCF Reed

Sur PCF_REED_0, les broches P0, P1 et P2 correspondent aux trois premiers contacts du premier DIP-switch et sont associées aux LED 1, 2 et 3 de PCF_LED_0. La broche P3 n'est pas exploitée. Les broches P4, P5 et P6 correspondent aux trois premiers contacts du deuxième DIP-switch et sont associées aux LED 4, 5 et 6 de PCF_LED_1. Enfin, la broche P7 n'est pas exploitée.

IV. Explication du programme

J'ai volontairement séparé le programme en petites fonctions. Cela permet de tester chaque partie seule avant de tout rassembler, et cela évite de répéter les mêmes instructions dans le main().

A. Configuration des GPIO — GPIO_InitPin()

Cette fonction prépare une broche du STM32. On lui donne le port (A, B ou C), le numéro de broche, son mode et le niveau demandé. Elle active l'horloge du port, modifie les registres CRL/CRH et utilise BSRR ou BRR pour fixer le niveau. Je l'ai réutilisée pour l'I^2C logiciel, le ZIF16 et l'USART.

B. Temporisation — delay_us() et delay_ms()

Les échanges I^2C ont besoin de petits temps d'attente pour que les signaux soient correctement reconnus. La fonction en microsecondes s'appuie sur SysTick. Pour attendre plus longtemps lors des essais LED, j'ai ajouté delay_ms(), qui appelle plusieurs fois la temporisation courte. Cela évite de dépasser la taille du compteur SysTick.

C. Gestion du ZIF16 — ZIF_Config() et ZIF_Lire()

Les pattes 1 à 8 du ZIF16 correspondent aux broches PC0 à PC7, et les pattes 9 à 16 aux broches PB8 à PB15. À partir de deux entiers de 16 bits, ZIF_Config() choisit pour chaque patte si elle doit être en entrée avec un rappel faible H/L ou en sortie push-pull imposant un niveau fort 0/1. Pour éviter les conflits, le code commence par libérer les broches avant de leur appliquer leur nouvelle configuration.

```
ZIF_Config(0xFFFFU, 0x0000U);  // 16 entrées, rappel bas
```

La fonction ZIF_Lire() récupère les registres GPIOC->IDR et GPIOB->IDR, puis les rassemble dans un entier de 16 bits. La fonction ZIF_DirectionsValides() limite à deux le nombre de sorties fortes, conformément au montage prévu pour l'alimentation du composant.

D. *I²C en bit banging — I2C_Start(), I2C_Stop(), I2C_EcrireBit(), I2C_LireBit()*

L'énoncé demande de reproduire l'I²C en logiciel, sans utiliser le périphérique I²C matériel. Le STM32 fait donc varier PB6 (SCL) et PB7 (SDA) lui-même. Ces fonctions servent à créer les conditions START et STOP, envoyer les bits du poids fort vers le poids faible et lire l'acquittement envoyé par le PCF8574.

E. *Dialogue avec les PCF8574 — PCF_Ecrire() et PCF_Lire()*

Les fonctions précédentes sont regroupées pour envoyer une adresse I²C et un octet, ou pour lire un octet provenant d'un PCF. PCF_Ecrire() est utilisé pour allumer les LED ; PCF_Lire() sert à récupérer l'état des entrées des interrupteurs. Le résultat de la fonction indique si le composant a bien répondu par ACK.

F. *Casiers — Casiers_Initialiser(), Casiers_LEDs() et Casiers_Lire()*

À l'initialisation, les PCF des LED sont mis dans un état éteint et le PCF des interrupteurs est libéré avec 0xFF afin de pouvoir lire les entrées. Casiers_LEDs() transforme les consignes rouge/vert en bits actifs à 0. Casiers_Lire() lit désormais un seul PCF de capteurs : son octet est placé dans les 8 bits faibles d'une variable 16 bits, pour garder le format de réponse du TP.

G. *USART2 et réception des trames — UART_Init() et USART2_IRQHandler()*

La liaison série utilise PA2 en transmission et PA3 en réception. Elle est configurée à 9600 bauds, 8 bits, sans parité, avec un bit de stop (8N1). Dès qu'un octet est reçu, l'interruption USART2_IRQHandler() lit son contenu et vérifie aussi la présence éventuelle d'erreurs de réception.

La machine à états MaeRcp distingue les commandes de casiers (en-tête 0xC0) de celles du ZIF16 (en-tête 0x80). Elle récupère ensuite les quatre octets de données, dont les bits de poids fort sont reconstitués à partir de l'en-tête. Une nouvelle en-tête peut servir à se resynchroniser.

H. *Transmission — FIFO_AjouterReponse() et FIFO_Emettre()*

Le STM32 ne renvoie pas les données directement au milieu du traitement : il prépare une réponse de trois octets dans une FIFO de 16 cases. FIFO_Emettre() envoie ensuite les octets au fur et à mesure que l'USART est disponible. Cela permet de gérer l'envoi sans bloquer tout le programme.

I. *Séquence de démarrage des LED — Allumer_Toutes_LEDs(1)*

Juste avant d'entrer dans le while(1) principal, le programme appelle Allumer_Toutes_LEDs(1). Cette fonction m'a permis de vérifier rapidement que les deux PCF8574 répondaient et que les six LED bicolores pouvaient bien changer de couleur. Elle réalise une seule séquence de test, avec

environ une seconde entre chaque état. Ce n'est pas la même fonction que celle qui lit les DIP-switchs.

Pour chaque étape, la fonction écrit la même valeur sur les deux PCF : 0xFF pour tout éteindre, 0xAA pour allumer les voies rouges, puis 0x55 pour allumer les voies vertes. Les sorties des LED sont actives à 0. Cette animation se fait au démarrage, puis le programme attend les commandes envoyées par Processing.

- Pour 0xFF envoyé -> Toutes éteintes
- Pour 0xAA -> Les six LED passent au rouge
- Pour 0x55 -> les six LED passent au vert

J. Test local des DIP-switchs — Maj_LEDs_Casiers()

J'ai également prévu Maj_LEDs_Casiers() pour vérifier les interrupteurs sans passer par l'ordinateur. Cette fonction lit le PCF des capteurs à l'adresse 0x23 puis calcule les octets à envoyer aux deux PCF des LED. Les bits P0, P1 et P2 commandent respectivement les LED 1 à 3, tandis que P4, P5 et P6 commandent les LED 4 à 6. P3 et P7 sont lus, mais ils ne servent pas à choisir une couleur dans ce mode de test.

Comme les DIP-switchs relient une entrée à la masse lorsqu'ils sont fermés, un bit à 0 allume la couleur verte et un bit à 1 allume la couleur rouge. Par contre, dans le main() actif fourni avec le projet, l'appel à Maj_LEDs_Casiers() se trouve uniquement dans un ancien main() commenté. Les LED ne suivent donc pas automatiquement les DIP-switchs en permanence : après l'animation de démarrage, la commande normale des LED passe par les trames de Processing.

K. Boucle principale — main()

Le main() initialise d'abord les GPIO, le ZIF16, l'I^2C, les PCF et l'USART. Ensuite, il attend les commandes préparées par l'interruption, met à jour les LED ou le ZIF16, lit l'état demandé et prépare la réponse. Pour les premiers tests matériels seulement, j'ai aussi utilisé Maj_LEDs_Casiers() pour faire varier les LED directement à partir des DIP-switchs, sans Processing. Ce mode de test est distinct du fonctionnement final prévu dans l'énoncé.

L. Récapitulatif de toutes les fonctions de main.c

Pour être sûr de ne rien oublier, j'ai repris ici les fonctions présentes dans le fichier main.c. Cela permet de retrouver rapidement où se fait chaque opération, sans recopier les 649 lignes du programme dans le rapport.

Au total, le programme contient 28 fonctions déclarées, dont une fonction de test matériel non appelée par la version active : Maj_LEDs_Casiers().

Fonction	Partie	Rôle précis
GPIO_InitPin()	GPIO	Configure une broche (CRL/CRH, état initial, horloge port).
GPIO_Ecrire()	GPIO	Force 0 ou relâche 1 sur SCL/SDA via BSRR/BRR.
delay_us()	Temporisation	Attend avec le compteur SysTick ; délais du bus I²C.
delay_ms()	Temporisation	Appelle delay_us(1000) pour l'animation des LED.
ZIF_Config()	ZIF16	Définit direction et niveau/pull de chacune des 16 pattes.
ZIF_Lire()	ZIF16	Assemble GPIOC->IDR et GPIOB->IDR dans un mot de 16 bits.
ZIF_DirectionsValides()	ZIF16	Refuse une configuration imposant plus de deux sorties fortes.
I2C_Init()	I²C logiciel	Configure PB6/PB7 en open-drain relâché.
I2C_Start()	I²C logiciel	Génère START par SDA haut → bas avec SCL haut.
I2C_Stop()	I²C logiciel	Génère STOP par SDA bas → haut avec SCL haut.
I2C_EcrireBit()	I²C logiciel	Émet un bit avec une impulsion SCL.
I2C_LireBit()	I²C logiciel	Relâche SDA et lit sa valeur pendant SCL haut.
I2C_EcrireOctet()	I²C logiciel	Transmet 8 bits puis vérifie l'ACK.
I2C_LireOctet()	I²C logiciel	Lit 8 bits puis répond par un NACK.
PCF_Ecrire()	PCF8574	Écrit un octet à l'adresse I²C choisie.
PCF_Lire()	PCF8574	Lit un octet renvoyé par un PCF8574.
Casiers_Initialiser()	Casiers	Éteint les LED et libère les entrées du PCF des contacts.
LEDs_CalculerOctet()	Casiers	Convertit les masques de couleurs en bits actifs à 0.
Casiers_LEDs()	Casiers	Met à jour les deux PCF commandant les six LED.
Casiers_Lire()	Casiers	Lit P0–P7 et construit un état 16 bits.
UART_Init()	USART2	Configure PA2/PA3, 9600 bauds, 8N1 et l'interruption RX.
Reception_Reconstituer()	USART2	Restaure les bits de poids fort depuis l'en-tête.
USART2_IRQHandler()	USART2	Décode 5 octets et rend la commande prête à traiter.
FIFO_AjouterReponse()	USART2	Place 3 octets de réponse dans la FIFO circulaire.
FIFO_Emettre()	USART2	Envoie un octet si TXE est disponible.
Allumer_Toutes_LEDs()	Test LED	Anime éteint → rouge → vert avant while(1).
Maj_LEDs_Casiers()	Test local	Commande LED depuis DIP-switchs : appel seulement dans main commenté.
main()	Programme principal	Initialise puis traite les casiers, le ZIF16 et l'émission UART.

Figure 5 : Recapitulatif de toutes les fonctions

L'ancienne boucle main() présente en commentaire n'est pas exécutée. Je la conserve dans le projet car elle montre comment j'ai testé les LED directement à partir des DIP-switchs, mais elle ne remplace pas le main() qui traite les trames de Processing.

V. Configuration périphérique STM32

A. *GPIO*

Fonction	Broche(s)	Mode dans main.c	État ou particularité
I²C SCL	PB6	GPIO_OUTPUT_OD_2MHZ	Libérée à 1 ; horloge générée par logiciel
I²C SDA	PB7	GPIO_OUTPUT_OD_2MHZ	Libérée à 1 ; données et acquittements
USART2 TX	PA2	GPIO_AF_PP_2MHZ	État initial haut, liaison série
USART2 RX	PA3	GPIO_INPUT_FLOATING	Réception d'octets et interruption RXNE
ZIF16 en entrée	PC0–PC7 et PB8–PB15	GPIO_INPUT_PULL	ODR = 1 pull-up ; ODR = 0 pull-down
ZIF16 en sortie	mêmes broches	GPIO_OUTPUT_PP_2MHZ	Niveau fort 0/1 selon commande

Figure 6 : Configuration GPIO pour I2C

Paramètre	Configuration	Commentaire
Bus	PB6=SCL ; PB7=SDA	GPIO open-drain et rappels haut nécessaires
Délais élémentaires	delay_us(5)	Temporisations logicielles par étapes
Adresse PCF LED 0	0x20	Octet d'adresse écriture = 0x40
Adresse PCF LED 1	0x21	Octet d'adresse écriture = 0x42
Adresse PCF contacts	0x23	0x46 en écriture ; 0x47 en lecture
ACK	SDA = 0	Acquittement après envoi de chaque octet

Figure 7 : etat par défaut

Pour le ZIF16, l'état de sécurité initial est directions = 0xFFFF et niveaux = 0x0000. Toutes les pattes sont donc en entrée avec rappel faible vers le bas, et aucune sortie forte ne vient imposer un potentiel au composant dès le démarrage.

B. *Configuration USART2*

Paramètre	Valeur du code	Commentaire
Vitesse	9600 bauds	Constante VITESSE_UART
Trame	8N1	8 bits de données, aucune parité, 1 stop
TX / RX	PA2 / PA3	AF push-pull 2 MHz / entrée flottante
Interruptions	RXNEIE activé	USART2_IRQHandler() reçoit les octets
Registre BRR	(PCLK1 + 4800) / 9600	Calcul à partir de la fréquence réelle APB1
FIFO émission	16 octets	Fifo, IwFifo, IrFifo et FifoLibre

Figure 8 : Configuration USART2

VI. Configuration et vérification du fonctionnement du système

A. *Configuration des périphériques*

Pour assurer la communication avec les PCF8574, nous utilisons le bus I^2C en *bit banging*, avec les broches PB6 pour le signal SCL et PB7 pour le signal SDA. Ces deux broches sont configurées en sortie open-drain, avec des résistances de rappel au niveau haut pour permettre la communication.

Les LED bicolores sont commandées à l'aide des sorties P0, P2 et P4 pour le rouge, et P1, P3 et P5 pour le vert. Chaque LED possède donc deux sorties de commande et fonctionne avec une anode commune. Pour allumer une couleur, il suffit de mettre la sortie correspondante à l'état logique 0.

Pour les capteurs, une écriture préalable de 0xFF sur le PCF8574 permet de libérer ses huit broches afin de pouvoir lire l'état des interrupteurs. Les DIP-switchs sont reliés à la masse : lorsqu'un interrupteur est fermé, le PCF lit un niveau logique 0, tandis qu'un interrupteur ouvert correspond à un niveau logique 1.

La communication avec l'ordinateur passe par l'USART2, configuré à 9600 bauds au format 8N1, via le port USB du ST-LINK.

Le format 8N1 signifie :

- 8 : chaque donnée est transmise sur 8 bits.
- N (None) : aucun bit de parité n'est utilisé.

- 1 : un bit de stop marque la fin de la transmission.

À cela s'ajoute un bit de start permettant d'indiquer le début de chaque transmission. À 9600 bauds, la durée d'un bit est d'environ 104 µs.

Enfin, le testeur ZIF16 est initialisé avec une direction de 0xFFFF et un niveau de 0x0000. Cela permet de configurer les 16 broches en entrée avec une résistance de rappel vers le bas, afin d'éviter d'imposer des niveaux forts lors du démarrage du système.

B. Tests et validation des interrupteurs

Lors des premiers essais, j'avais branché les DIP-switchs entre VDD et les entrées du PCF8574. Cependant, je me suis rendu compte que les entrées du PCF disposent déjà d'un faible rappel au niveau haut lorsqu'elles sont libérées avec 0xFF.

Ainsi, lorsque je changeais la position des interrupteurs, la valeur lue restait à 0xFF, puisque les entrées étaient toujours au niveau haut. J'ai donc modifié le câblage afin de relier les interrupteurs à la masse, conformément au fonctionnement demandé dans le TP.

Pour vérifier la lecture, nous pouvons prendre l'exemple des deux interrupteurs connectés aux broches P0 et P1.

P0	P1	Valeur attendue
Ouvert	Ouvert	`0x00FF`
Fermé	Ouvert	`0x00FE`
Ouvert	Fermé	`0x00FD`
Fermé	Fermé	`0x00FC`

Ces valeurs correspondent aux états logiques attendus lorsque les autres entrées restent à 1. Chaque interrupteur fermé force son entrée au niveau bas, ce qui fait passer le bit correspondant de 1 à 0.

Ces valeurs restent théoriques et devront être comparées aux résultats réellement obtenus à l'aide du débogueur Keil ou du logiciel Processing.

C. *Vérification des niveaux logiques du ZIF16*

Pour vérifier le fonctionnement du testeur ZIF16, nous avons utilisé le logiciel Processing, qui permet d'envoyer les configurations des broches au STM32 et de récupérer leurs états logiques.

Lors d'un des essais, nous avons obtenu la valeur hexadécimale 0x7FFF.

En binaire, cette valeur correspond à :

0111 1111 1111 1111

On constate donc que les quinze premières broches sont lues à l'état logique 1, tandis que la broche 16, correspondant à PB15, est à l'état logique 0.

Cette lecture semble cohérente avec la configuration affichée dans Processing, puisque la dernière broche est configurée avec un niveau bas. Elle montre également que les données reçues peuvent être interprétées individuellement pour chacune des 16 broches.

Toutefois, pour confirmer complètement le fonctionnement du testeur, il faudra comparer les niveaux lus avec ceux réellement appliqués au niveau du connecteur ZIF16, ce qui n'a pas été fait faut d'avoir un oscilloscope avec un grand nombre de voix.

VII. Le logiciel processing

Le logiciel Processing envoie les consignes à la carte NUCLEO et permet de visualiser les réponses. L'interface propose seize cases avec des voyants rouge et vert, ainsi qu'un dessin du composant ZIF16. Même si l'écran représente seize casiers, seuls les six premiers voyants et une partie des contacts correspondent au matériel disponible dans notre maquette.

Dans la fenêtre, les 16 emplacements sont répartis sur deux colonnes de huit cases. La colonne de gauche correspond aux indices 0 à 7 du tableau tableau_capteur[], donc aux broches P0 à P7 du PCF des interrupteurs. La colonne de droite représente les indices 8 à 15, mais sur la maquette actuelle aucun deuxième PCF de capteurs n'est utilisé : ces cases sont donc renseignées à 0 par la réponse sur 16 bits. Les six voyants réellement câblés sont les six premières cases de la colonne de gauche.

La valeur au centre de chaque case représente l'état lu du capteur : 0 si le DIP-switch est fermé vers la masse, 1 s'il est ouvert. Les ronds rouge et vert, eux, servent à préparer les commandes d'allumage. Modifier un rond ne suffit pas à envoyer la commande : il faut ensuite appuyer sur C. De la même façon, une action sur un DIP-switch n'est affichée qu'après une nouvelle commande de lecture des casiers.

Les commandes sont envoyées au STM32 sous la forme de cinq octets. Le bit 7 vaut 1 uniquement dans l'en-tête, tandis que les bits de poids fort des quatre octets suivants sont conservés dans les bits 0 à 3 de cet en-tête.

Octet	Commande casiers	Commande ZIF16
0	En-tête 0xC0 + bits 7	En-tête 0x80 + bits 7
1	LED vertes, poids fort	Directions PB15–PB8
2	LED vertes, poids faible	Directions PC7–PC0
3	LED rouges, poids fort	Niveaux PB15–PB8
4	LED rouges, poids faible	Niveaux PC7–PC0

Figure 9 : Commandes STM32 envoyés

Après avoir traité la commande, le STM32 renvoie trois octets : l'en-tête (0xC0 pour la lecture des contacts ou 0x80 pour le ZIF16), puis l'octet de poids fort et celui de poids faible de la valeur lue. Processing reconstitue ensuite le mot de 16 bits.

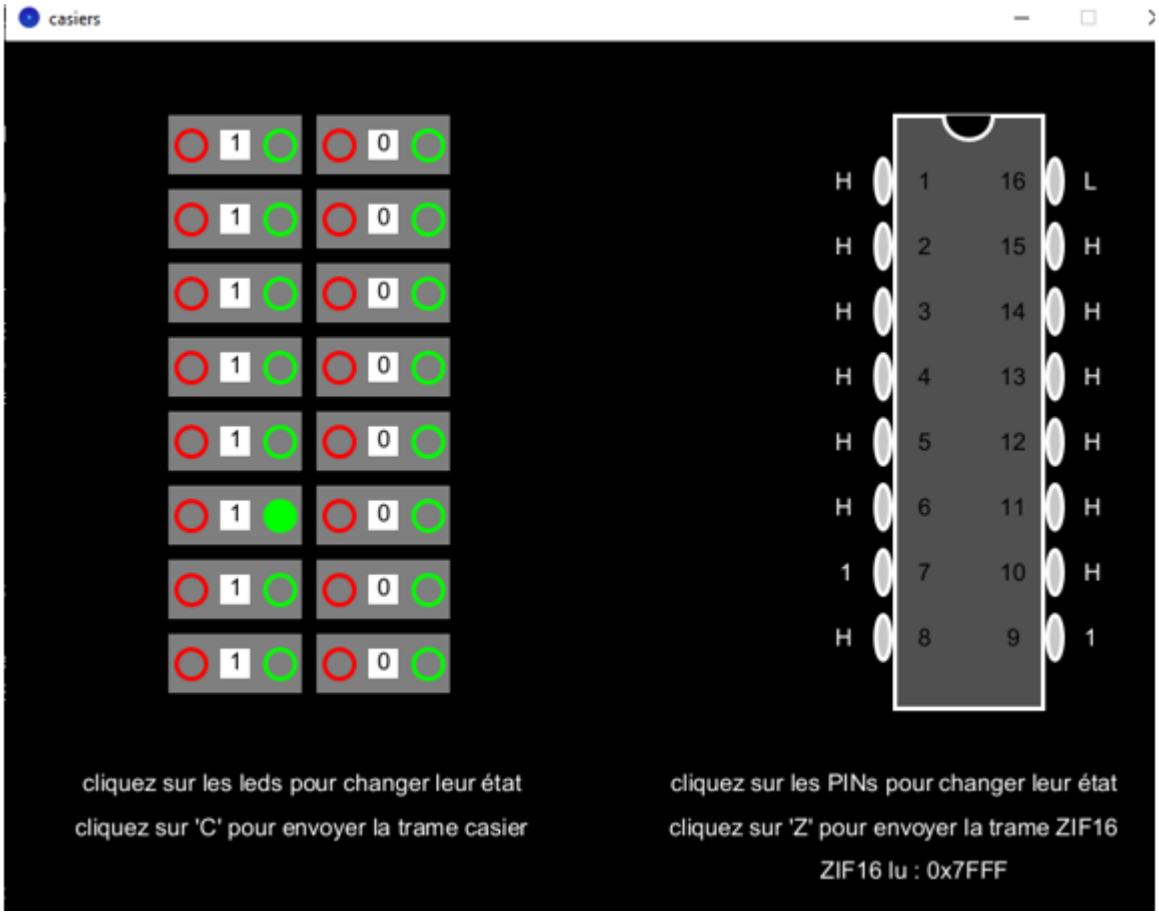


Figure 10 : Capture de processing : commande des casiers et valeurs ZIF16

VIII. Petite explication concernant l

A. *DIP-switchs*

Lors des premiers essais, j'avais relié les interrupteurs au niveau haut. Comme les entrées du PCF8574 sont déjà faiblement rappelées à 1 lorsqu'elles sont libérées, je récupérais toujours 0xFF. J'ai donc modifié le câblage pour que les interrupteurs ferment le contact vers la masse.

La capture Processing ajoutée ci-dessous permet cette fois de repérer concrètement les boutons dans l'interface. Le code casiers.pde place les indices 0 à 7 en haut et en bas de la colonne de gauche ; j'ai donc repris ce positionnement dans le tableau suivant.

P0	P1	Valeur attendue	Explication
Ouvert	Ouvert	0x00FF	Aucun contact à la masse
Fermé	Ouvert	0x00FE	Bit 0 forcé à 0
Ouvert	Fermé	0x00FD	Bit 1 forcé à 0
Fermé	Fermé	0x00FC	Bits 0 et 1 forcés à 0

Dans cette capture Processing, les cases 4 et 8 de la colonne de gauche affichent 1, tandis que les autres affichent 0 : cela correspond à 0x88 pour les huit entrées visibles. Le nouveau chronogramme I^2C montre, lui, une lecture 0x98, qui ajoute P4=1. Les deux captures ne documentent donc pas exactement la même lecture ; on ne peut pas les considérer comme parfaitement synchronisées.

Entrée PCF	Case dans Processing	Correspondance test LED
P0	Gauche, ligne 1 (casier 1)	LED 1
P1	Gauche, ligne 2 (casier 2)	LED 2
P2	Gauche, ligne 3 (casier 3)	LED 3
P3	Gauche, ligne 4 (casier 4)	Non utilisée dans Maj_LEDs_Casiers()
P4	Gauche, ligne 5 (casier 5)	LED 4
P5	Gauche, ligne 6 (casier 6)	LED 5
P6	Gauche, ligne 7 (casier 7)	LED 6
P7	Gauche, ligne 8 (casier 8)	Non utilisée dans Maj_LEDs_Casiers()

Attention : le numéro de case correspond à l'indice lu, tandis que la numérotation physique des six LED est celle utilisée par les deux PCF de sortie. Les deux notions ne sont pas entièrement identiques, notamment à partir de P4.

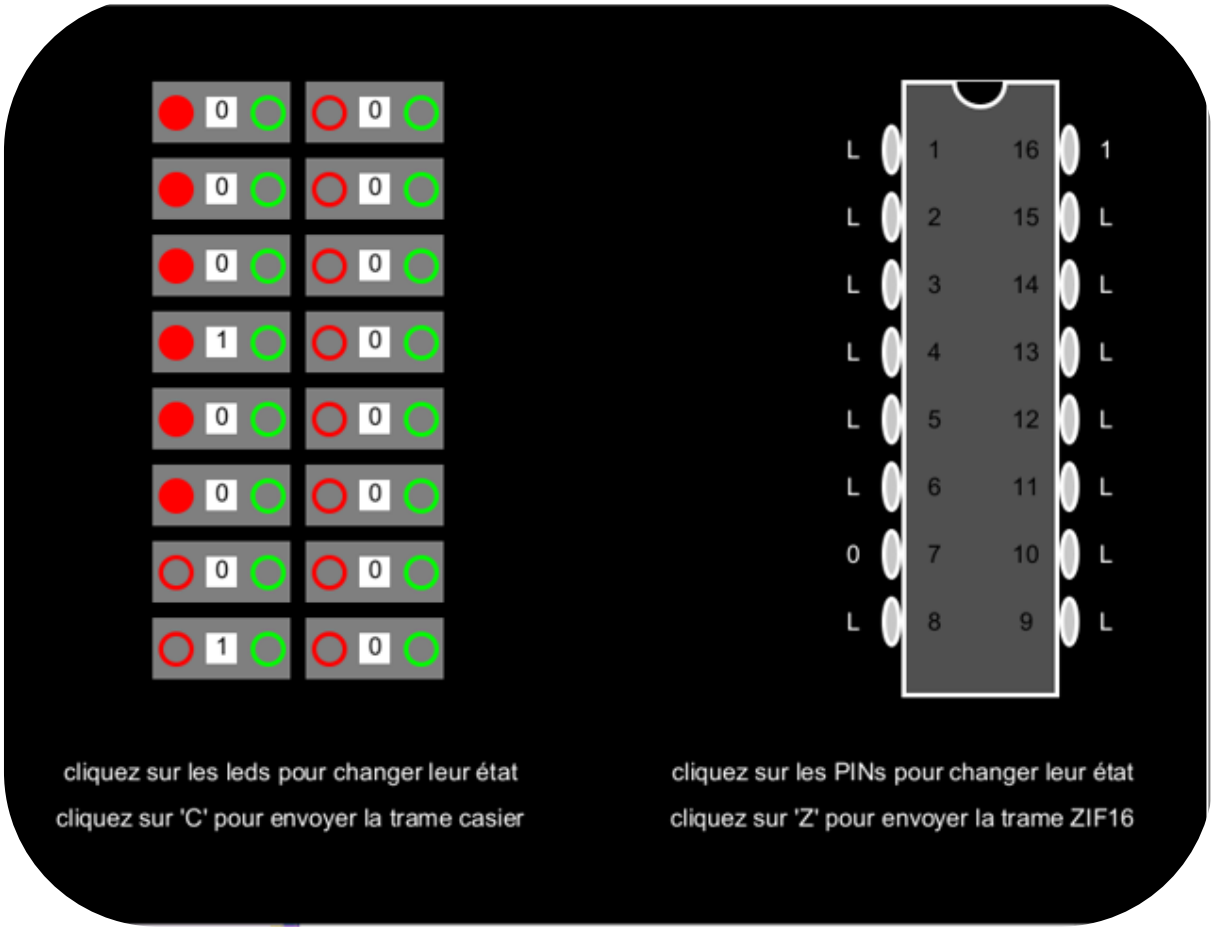


Figure 11 : Fenêtre Processing fournie : cases de gauche liées à P0–P7 et six voyants rouges commandés

B. Première lecture et vérification du ZIF16

Sur une première capture Processing réalisée pendant les essais, on pouvait lire 0x7FFF. Cette valeur correspond à une patte 16 à 0 et aux quinze autres pattes à 1. Cette lecture reste utile pour comprendre le principe du mot de 16 bits. Une nouvelle acquisition présentée dans la partie IX permet maintenant de vérifier un autre exemple complet, 0x6E97, avec la trame UART correspondante.

Le STM32 reconstruit les états à partir des registres GPIOC->IDR (PC0 à PC7) et GPIOB->IDR (PB8 à PB15). La correspondance logique est vérifiée grâce aux nouveaux octets reçus, mais une validation électrique complète demanderait encore de mesurer physiquement plusieurs pattes du support ZIF16 pendant leur changement de niveau.

IX. Commandes des LED et lectures des DIP-switchs

Dans ce premier essai, j'ai voulu vérifier toute la chaîne de communication, depuis un clic sur les LED de Processing jusqu'aux PCF8574. Le logiciel envoie une commande sur l'USART2. Le STM32 la décode, met à jour les LED par I²C, lit les DIP-switchs, puis renvoie leur état au PC. Grâce à PulseView, on peut observer la partie série et la partie I²C dans la même acquisition.

A. *Commande envoyée depuis Processing*

Sur la capture, les six premières LED sont demandées en rouge. Le masque des couleurs rouges vaut 0x003F, tandis que celui des LED vertes vaut 0x0000. Avec le protocole de casiers prévu dans casiers.pde, la commande complète envoyée sur UART est C0 00 00 00 3F.

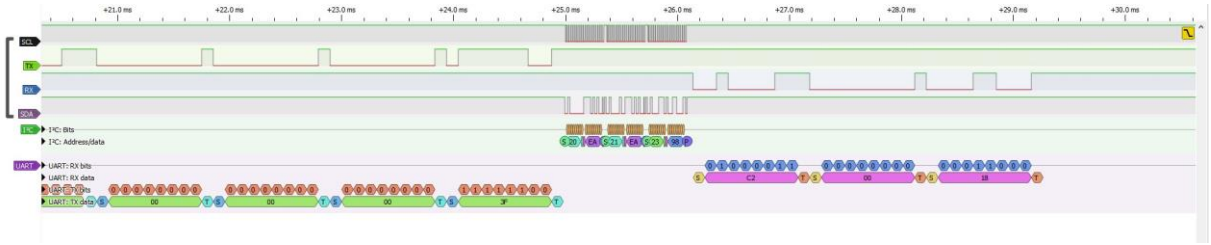


Figure 12 : Commande UART des casiers : C0 00 00 00 3F, suivie du traitement I²C

Octet	Valeur	Utilisation
1	C0	En-tête de commande casiers.
2	00	LED vertes : octet fort.
3	00	LED vertes : octet faible.
4	00	LED rouges : octet fort.
5	3F	LED rouges : octet faible (six LED à 1).

Les intitulés TX et RX affichés par l'analyseur sont ceux donnés aux voies lors du branchement. Dans cette prise de mesure, TX représente le sens de la commande provenant du PC, alors que la réponse du STM32 apparaît sur RX. Au niveau des broches du STM32, c'est bien PA3 qui reçoit et PA2 qui transmet.

B. *Commandes et lecture sur le bus I²C*

Après réception des cinq octets, Casiers_LEDs() appelle LEDs_CalculerOctet() pour préparer l'octet à écrire sur les deux PCF. Comme les sorties des LED sont actives à 0, chaque LED rouge allumée doit recevoir un 0 sur sa sortie rouge et un 1 sur sa sortie verte. On obtient alors 0xEA, soit 1110 1010 en binaire.

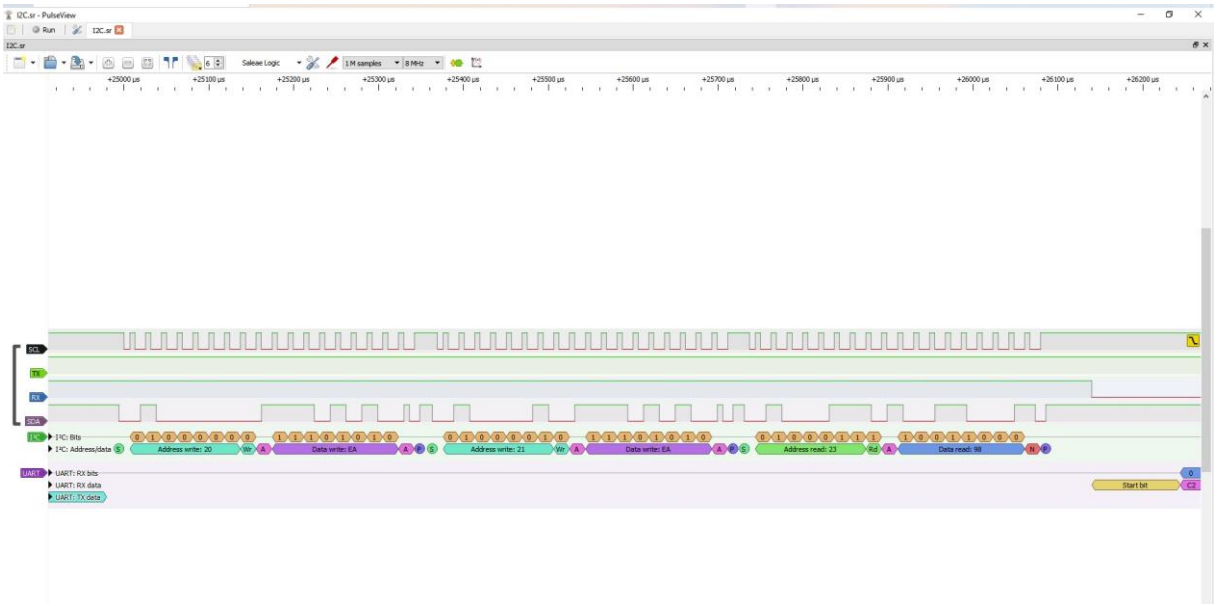


Figure 13 : Capture globale du bus I²C : commandes des LED puis lecture des contacts.

Transaction	Octet lu/écrit	Rôle dans le montage
0x20 — écriture	EA	Premier PCF8574 : LED 1 à 3 en rouge.
0x21 — écriture	EA	Deuxième PCF8574 : LED 4 à 6 en rouge.
0x23 — lecture	98	État lu sur P0 à P7 des DIP-switchs.

Sur les deux écritures, PulseView affiche un ACK après l'adresse et après la donnée : les PCF répondent donc à la commSande. Ensuite, la lecture sur 0x23 renvoie 0x98. En binaire, cette valeur vaut 1001 1000 : P3, P4 et P7 sont à 1 (interrupteurs ouverts) ; P0, P1, P2, P5 et P6 sont à 0 (interrupteurs fermés vers la masse). Un NACK termine normalement la lecture du dernier octet.



Figure 14 : Zoom temporel : liaison UART, puis démarrage des échanges I²C.

La sortie UART du STM32 renvoie la valeur lue en trois octets : C2 00 18. Ici C2 est l'en-tête de réponse C0 avec le bit 7 de l'octet faible remonté dans le bit 1 ; les octets de données 00 et 18 contiennent les sept bits faibles. Processing reconstitue alors 0x0098. Sur une autre capture de la fenêtre, on lit 0x0088 : P4 n'a pas le même état. Ces captures ne sont donc pas strictement simultanées, ce qu'il faut préciser.

C. *Correspondance avec les commandes du code*

Cette séquence confirme le rôle des fonctions USART2_IRQHandler(), Reception_Reconstituer(), Casiers_LEDs(), PCF_Ecrire(), PCF_Lire(), Casiers_Lire(), FIFO_AjouterReponse() et FIFO_Emettre(). Le changement demandé sur Processing entraîne bien une écriture sur les sorties des PCF, puis la lecture des interrupteurs peut être affichée dans la même interface. Le fait que les LED soient rouges sur la capture du PC ne suffit pas à lui seul à prouver leur état physique : la vidéo permettra de montrer les six LED sur la maquette.

X. Configuration et lecture des 16 broches du ZIF16

Dans ce second essai, le fonctionnement est différent : il n'y a plus de PCF8574 à commander. Processing transmet directement une configuration des 16 broches du support ZIF. Le STM32 reconfigure alors les GPIO reliés au support et renvoie la valeur réellement lue sur leurs entrées. C'est cette valeur que l'on voit en bas à droite de Processing.

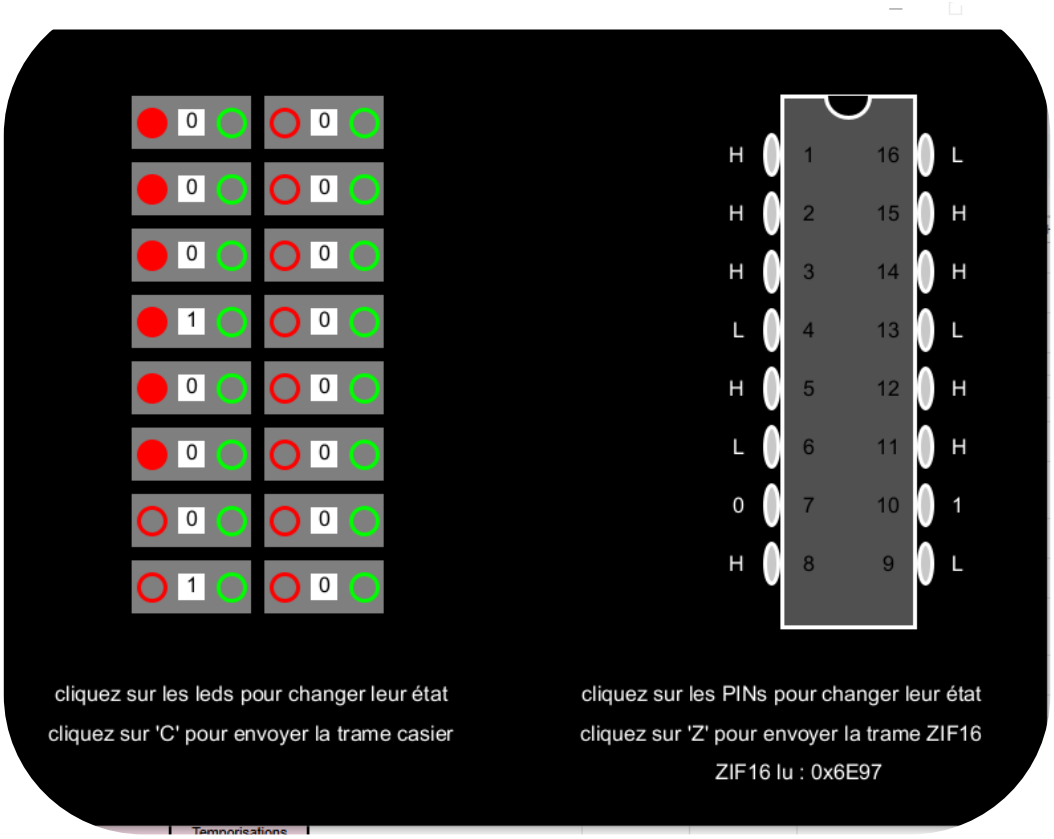


Figure 15 : Fenêtre Processing avec les niveaux H/L du ZIF16 et le résultat 0x6E97

Sur cette nouvelle capture, Processing affiche « ZIF16 lu : 0x6E97 ». Le mot binaire correspond à 0110 1110 1001 0111. Les pattes 1 à 8 sont reliées à PC0–PC7, et les pattes 9 à 16 à PB8–PB15 : chaque bit du mot correspond donc à l'une des pattes, la patte 1 étant le bit 0. Les valeurs H et L affichées sur la représentation du support sont cohérentes avec ce mot reçu.



Figure 16 : Commande ZIF16 de cinq octets et réponse UART sur trois octets.

Sens	Octets visibles	Interprétation
PC → STM32	8B 7D 3F 6E 17	Commande ZIF : directions et niveaux demandés, avec bits 7 reportés dans 8B.
STM32 → PC	82 6E 17	Réponse ZIF : état lu 0x6E97 reconstitué à partir de l'en-tête.

Pour décoder la commande, le programme replace les quatre bits de poids fort contenus dans l'en-tête 0x8B : le mot directions vaut alors 0xFDBF et le mot niveaux vaut 0x6E97. Avec directions = 0xFDBF, seules les pattes 7 et 10 sont configurées en sorties fortes ; les quatorze autres restent en entrée avec rappel haut ou bas suivant le bit correspondant du mot niveaux. Cela respecte la limite de deux sorties imposée par ZIF_DirectionsValides().

Le STM32 applique ensuite cette configuration avec ZIF_Config(), attend 20 µs, puis lit le mot ZIF_Lire(). Dans cet exemple, il récupère 0x6E97. Pour ne pas envoyer d'octets de données dont le bit 7 vaut 1, FIFO_AjouterReponse() transforme cette valeur en 82 6E 17. Le bit 1 de 0x82 permet de redonner le bit 7 de l'octet faible 0x97. Processing refait l'opération inverse et affiche 0x6E97.

Cette capture montre que le protocole UART fonctionne dans les deux sens et que la valeur de 16 bits est correctement reconstruite dans Processing. En revanche, elle ne suffit pas, à elle seule, à prouver électriquement les niveaux de chaque patte du ZIF16. Pour être totalement conforme aux essais demandés par le professeur, il faudra compléter avec une mesure sur les GPIO ou une vidéo montrant une patte pilotée en H/L puis forcée depuis le câblage.

XI. Difficultés rencontrées et assistance IA

Globalement, comme pour le TP2, je n'ai plus tout l'historique de mes conversations avec l'IA, mais en gros, on débattait pas mal. Je lui expliquais comment je voulais faire les choses, selon ma vision, puis elle me disait parfois que ce n'était pas forcément ce qui était demandé dans le TP. J'essayais alors de comprendre pourquoi, et on revenait sur les différentes solutions possibles.

Au début, mes prompts étaient assez bateau. Je donnais très peu d'informations, en mode « Fais-moi cette fonction » ou « Pourquoi mon code ne marche pas ? ». Forcément, les réponses n'étaient pas toujours adaptées à ce que je voulais faire. Petit à petit, j'ai compris qu'il fallait être plus précis, notamment en donnant les fonctions déjà écrites, les broches utilisées et le résultat que je cherchais à obtenir.

Par exemple, j'ai utilisé l'IA pour m'aider à comprendre la communication I^2C en *bit banging*, le fonctionnement des PCF8574 et la manière de reconstruire les trames UART. On revenait aussi

régulièrement sur les différences entre ce que je pensais que le programme faisait et ce qu'il faisait réellement.

Une des difficultés rencontrées concernait notamment les DIP-switchs. Au départ, je les avais reliés au niveau haut, sauf que je récupérais toujours 0xFF lors des lectures. Après avoir repris le fonctionnement des entrées du PCF8574, j'ai compris qu'il fallait les relier à la masse pour pouvoir observer correctement les changements d'état.

J'ai également rencontré quelques difficultés avec la gestion du ZIF16, notamment pour comprendre comment configurer les broches en entrée ou en sortie et reconstruire les valeurs sur 16 bits. L'IA m'a surtout servi à reprendre les explications étape par étape, pour comprendre ce qui se passait réellement dans les registres.

Au final, je ne me suis pas contenté de récupérer du code généré. J'ai essayé de comprendre les fonctions et de comparer les résultats avec ce que j'observais sur Processing et l'analyseur logique. C'est aussi ce qui m'a permis de mieux structurer mes demandes et d'être plus autonome au fur et à mesure du TP.

Exemples de prompts utilisés ou reformulés :

- « Explique-moi comment fonctionne l'I²C en bit banging sur le STM32 et à quoi servent les conditions START, STOP et ACK. »
- « Pourquoi je récupère toujours 0xFF lorsque je lis mes DIP-switchs avec le PCF8574 ? »
- « Explique-moi comment les quatre octets reçus par UART permettent de configurer les 16 broches du ZIF16 et comment les bits de poids fort sont reconstitués. »

Ces exemples sont reformulés, car je n'ai pas conservé les formulations exactes de toutes mes anciennes conversations.

XII. Conclusion

Pour conclure, ce TP m'a permis de mieux comprendre comment faire communiquer plusieurs composants autour d'un même microcontrôleur. Le STM32F103RB devait à la fois gérer les LED des casiers, lire les DIP-switchs, configurer les broches du ZIF16 et échanger des informations avec Processing.

Ce qui m'a surtout intéressé, c'était de pouvoir suivre le cheminement des données, depuis une commande envoyée sur l'ordinateur jusqu'à sa transmission sur le bus I²C, puis de retrouver les

réponses envoyées par le STM32. Grâce à l'analyseur logique, j'ai pu vérifier les trames et mieux comprendre le fonctionnement des protocoles UART et I^2C.

J'ai également pu tester la configuration du ZIF16 et retrouver dans Processing les valeurs reconstituées à partir des trames reçues. Même si les échanges UART ont pu être vérifiés, il resterait intéressant de contrôler physiquement davantage de broches pour confirmer complètement les niveaux appliqués au support.

Ce TP m'a aussi permis de progresser en programmation C, notamment dans la manipulation des registres du STM32 et la gestion des communications. J'ai rencontré plusieurs difficultés, surtout au niveau du câblage et de l'interprétation de certaines valeurs, mais les différents essais m'ont permis de mieux comprendre les erreurs et de les corriger progressivement.

Au final, même si tout n'a pas forcément fonctionné du premier coup, ce TP m'a permis de mieux faire le lien entre le code, les communications et le fonctionnement réel du montage, ce qui était pour moi l'un des objectifs les plus importants.

Tables des figures

Figure 1 : Visualisation Keil µVision + Visual Studio Code 6
Figure 2 : Bilan E/S - Carte Nucleo STM32 F103RB 7
Figure 3 : Localisation des composants et configuration des modules 8
Figure 4 : Câblage spécifier du PCF Reed 8
Figure 5 : Recapitulatif de toutes les fonctions 12
Figure 6 : Configuration GPIO pour I2C 13
Figure 7 : etat par défaut 13
Figure 8 : Configuration USART2 14
Figure 9 : Commandes STM32 envoyés 17
Figure 10 : Capture de processing : commande des casiers et valeurs ZIF16 17
Figure 11 : Fenêtre Processing fournie : cases de gauche liées à P0–P7 et six voyants rouges commandés 19
Figure 12 : Commande UART des casiers : C0 00 00 00 3F, suivie du traitement I^2C 20
Figure 13 : Capture globale du bus I^2C : commandes des LED puis lecture des contacts. 21
Figure 14 : Zoom temporel : liaison UART, puis démarrage des échanges I^2C. 22
Figure 15 : Fenêtre Processing avec les niveaux H/L du ZIF16 et le résultat 0x6E97 23
Figure 16 : Commande ZIF16 de cinq octets et réponse UART sur trois octets. 23